KTHA: Serverless Node.js App Host

Design Document

1. Overview

KTHA is a serverless host for Node.js applications. It accepts HTTP requests, routes them to isolated guest apps by path prefix, and manages the app lifecycle: cold-start on first request, reverse-proxy while active, shutdown after idle timeout.

The system is split into two binaries. `ktha-node` is the host process: it runs the reverse proxy, manages app lifecycles, and exposes an admin API. `ktha-runner` is the container runtime: it sets up Linux namespaces and cgroups, prepares the filesystem, and execs the Node.js process. The host spawns a runner for each container.

Go was chosen for the host: it provides direct access to Linux syscalls for namespace and cgroup setup, has good concurrency primitives for managing multiple containers, and produces a single static binary with low per-process overhead.

2. Image Format

The image format defines the contract between the host and the guest app. An image is a directory containing:

- `index.js` — the application entrypoint, a single bundle produced by esbuild from the app's TypeScript source and all its dependencies. Bundling eliminates `node_modules` from the image entirely: no package manager runs on the host, and the container filesystem stays minimal.
- `public/` (optional) — static assets loaded at runtime.

The guest app communicates with the host over a Unix domain socket. The runner sets the `KTHA_SOCK` environment variable to the socket path; the app creates an HTTP server listening on it. Unix sockets avoid port allocation and exhaustion, and since the container has no network namespace (`CLONE_NEWNET`), there is no TCP stack to expose. The socket file lives inside the container rootfs, naturally scoping it to a single container.

Readiness is determined by an HTTP healthcheck. The runner polls `GET /healthcheck` on the socket every 100ms. The app must respond with status 200 and `{"ok": true}` once it is ready to serve traffic. This gives the app control over when it declares itself ready — unlike checking for socket file existence, which only indicates that the process has called `listen()`, not that routes are registered or initialization is complete. The overall readiness timeout is 30 seconds.

3. Runner and Containerization

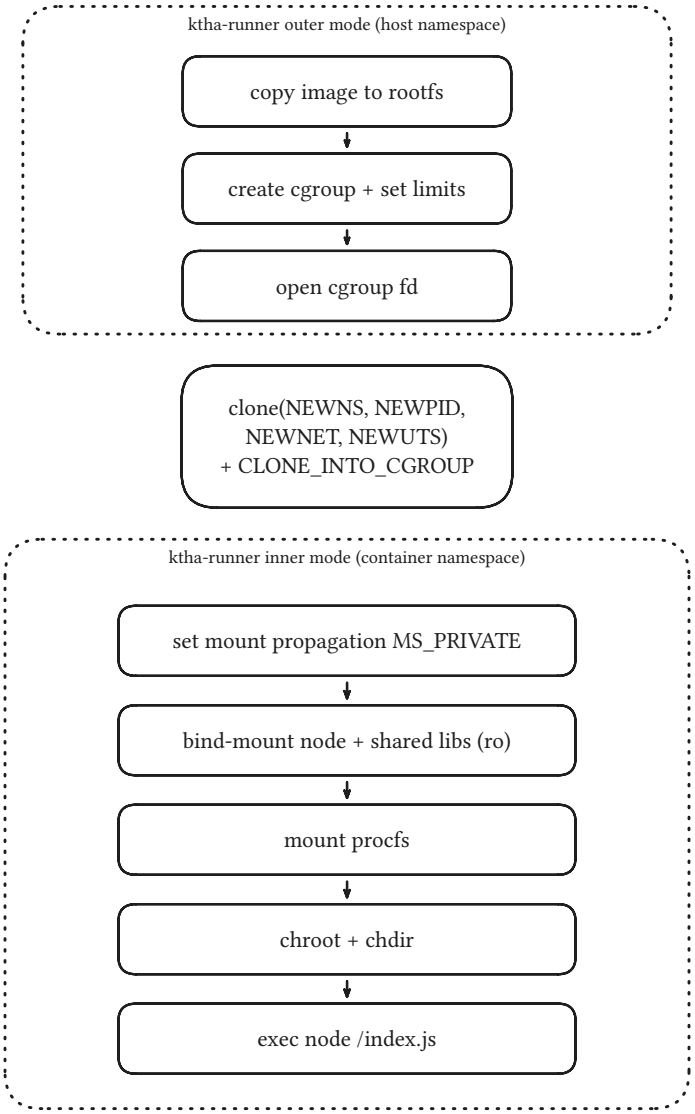

Figure 1: Container internals: runner outer/inner self-reexec across the namespace boundary.

The runner (`ktha-runner`) creates an isolated environment for the guest app using Linux namespaces and cgroups v2.

3.1. Container Environment

From the guest's perspective, the container has:

- A **PID namespace** — the runner in inner mode runs as PID 1, with the Node.js process as its only child.
- No **network** — `CLONE_NEWNET` isolates the network stack entirely. Communication happens over the Unix socket.
- A separate **hostname** (`CLONE_NEWUTS`), set to the container ID.
- A **chrooted filesystem** containing only the image files, the Node.js binary, and system libraries (`/lib`, `/lib64`) bind-mounted read-only from the host. A `procfs` is mounted for introspection.

Resource limits are enforced via **cgroups v2**: memory ceiling, PID count, and CPU time quota. The cgroup is created and configured from the host side before the process enters the namespace, and the process is placed into it at clone time via file descriptor (`CLONE_INTO_CGROUP`). `memory.oom.group` is set to 1, so an OOM event kills the entire cgroup cleanly rather than picking individual victims.

Mount propagation is set to `MS_PRIVATE | MS_REC` before any bind-mounts are performed, preventing any mounts inside the container from propagating back to the host.

3.2. Outer/Inner Self-Reexec

Container setup requires operations on both sides of the namespace boundary. The runner handles this via self-reexec: it runs in two modes within a single binary.

In **outer mode**, the runner performs host-privileged work: copies the image to the container rootfs, creates and configures the cgroup, opens the cgroup file descriptor. It then re-executes itself with an `--inner` flag, passing `CLONE_NEWNS | CLONE_NEWPID | CLONE_NEWNET | CLONE_NEWUTS` as clone flags.

In **inner mode**, the runner is already inside the new namespace. It sets mount propagation to private, bind-mounts the Node.js binary and system libraries as read-only, mounts `procfs`, performs `chroot`, and execs `node /index.js`.

This approach is inspired by Liz Rice's work on containers from scratch.

3.3. A Note on Node.js RSS

A minimal Node.js echo server reports ~50 MB of RSS. Investigation shows that most of this is shared pages from the Node.js binary and system libraries that are bind-mounted into the container. These pages are backed by the same files across all containers, but cgroup memory accounting charges them to whichever container's process faults them in first — introducing uncertainty into per-tenant memory consumption. This observation informs a design decision in the host process, described in the next section.

4. Host Architecture

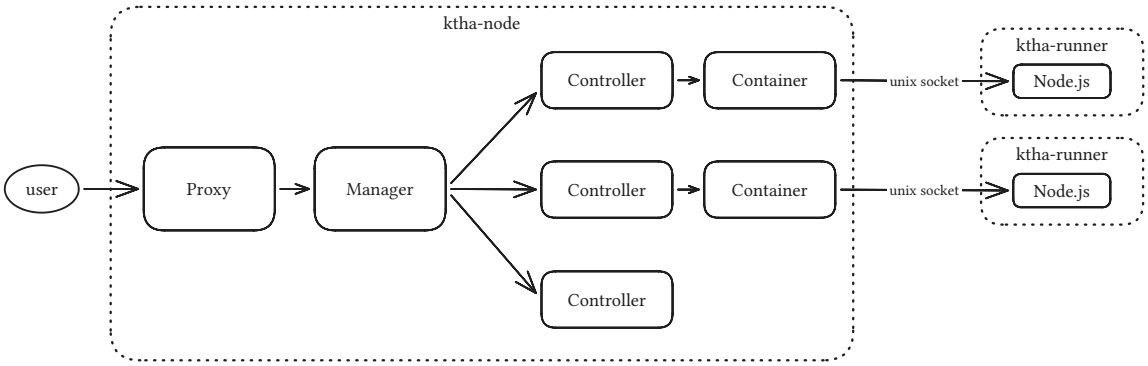

Figure 2: Request flow through ktha-node: proxy routes to manager, which fans out to per-app controllers.

`ktha-node` is the host process. It is structured as three layers: container, controller, and manager — each with a distinct responsibility.

4.1. Page Cache Warmer

On startup, `ktha-node` spawns a dummy Node.js process. This pre-populates the page cache with shared pages from the Node.js binary and system libraries, so that subsequent containers do not pay the cost of faulting these pages in, and the pages are not charged to any guest's cgroup memory accounting.

4.2. Container

The `container` package wraps a single runner process for the duration of its lifetime. It translates app-level configuration (image path, resource limits, environment variables) into runner CLI flags, starts the process, and exposes handles to the caller: the host-side Unix socket path and cgroup resource metrics.

The container performs the readiness check — it is a per-process-lifetime concern, executed exactly once for each new runner process.

Graceful shutdown is handled at this layer: the container tracks inflight requests via a reference counter. When a stop is requested, it waits for inflight requests to drain (with a configurable timeout), then sends SIGTERM to the runner. If the process does not exit within a stop timeout, it sends SIGKILL.

4.3. Controller

The controller represents a single app as the user sees it. It starts as an inert data structure — no container is running until the first request arrives.

On the first `Dial()`, the controller spawns a container and arms an idle timer. Subsequent dials reset the timer. When the timer fires, the controller gracefully stops the container — scaling to zero.

The controller also handles app upgrades. The current policy is lax: it updates the image path, then stops the old container asynchronously. New requests cold-start on the new image; inflight connections on the old container are drained. This creates a brief window where two instances of the same app may exist. A stricter stop-first policy would introduce downtime; a start-first policy with a readiness gate would provide honest zero-downtime upgrades. Both are straightforward to implement but were not required for the prototype.

4.4. Manager

The manager is a map of controllers behind a mutex. It routes incoming dial requests to the correct controller by app ID, and handles adding and removing apps at runtime. It is the entry point for the admin API's app management endpoints.

5. Reverse Proxy

The reverse proxy handles user-facing HTTP traffic. It extracts the app ID from the request path (`/app-id/...`), overrides the `DialContext` function on `httputil.ReverseProxy` to call through the manager, and forwards the request over the Unix socket to the guest app.

Cold starts are transparent to the caller: the dial blocks until the container is ready. The proxy distinguishes cold and warm starts and records both in the request metrics.

A metering middleware wraps the proxy handler, recording request duration per app, labeled by cold/warm start and dial success. These metrics are described in the Observability section.

6. Admin API

The admin API runs on a separate port from the user-facing proxy, providing a clean split between guest traffic and management operations.

Endpoints:

- **Upgrade** — update an app's image and restart the container.
- **Add / Delete** — register or remove apps at runtime. These are beyond the scope of the assignment, but proved invaluable during load testing for dynamically managing the app pool.

The admin API also serves the Prometheus metrics endpoint.

7. Observability

Metrics are structured outside-in, mirroring a natural debugging flow:

- **Proxy metrics** (what the user sees): request duration histograms per app, labeled by cold/warm start and dial success. These give RED (Rate, Errors, Duration) signals for each app.
- **Container metrics** (what the app consumes): memory usage, CPU time, and PID count read from cgroup v2 files. Each container exposes these to the host via the container layer.
- **Host metrics** (is the host the bottleneck): Go runtime metrics (goroutine counts, GC pauses, open file descriptors) and host-level I/O throughput read from the host's own cgroup — standard Go/process collectors do not expose disk I/O, so a separate cgroup-based collector was added.

All metrics are gathered into a single Prometheus registry and exposed via the admin API.

8. Design Alternatives

The isolation approach sits on a spectrum with three tiers:

Application-level isolation (V8 Isolates, as used by Cloudflare Workers). Multiple tenants run in isolated V8 heaps within a single process. Cold starts are near-instant (microseconds), memory overhead is minimal. However, isolation is at the VM level — a V8 exploit compromises all tenants. More importantly, the runtime environment is not Node.js: there is no access to `node:http`, `node:fs`, or other built-in modules. This makes it unsuitable for running standard Node.js applications.

OS-level isolation (Linux namespaces and cgroups — this project). Each app runs in its own process with its own PID, mount, network, and UTS namespaces. The app has access to the full Node.js runtime. Isolation is enforced by the kernel. The tradeoff is higher per-container overhead (~5–10 MB of unique memory after accounting for shared pages) and cold start in the hundreds of milliseconds.

VM-level isolation (Firecracker microVMs, as used by AWS Lambda). Each tenant gets a lightweight virtual machine with a dedicated kernel. Strongest isolation boundary — a guest kernel exploit does not affect the host. Highest overhead and startup cost.

KTHA uses OS-level isolation because the goal is to run standard Node.js applications with real filesystem and process semantics, at a cost appropriate for a single-host prototype.

8.1. Drawbacks

The primary drawback is the `CAP_SYS_ADMIN` requirement. The host process needs root privileges to create namespaces, set up cgroups, and perform mounts. This makes it difficult to run `ktha-node` inside a container: it requires privileged mode, cgroup namespace delegation, and a permissive seccomp profile — which largely negates the benefit of containerizing the host in the first place.

The pragmatic answer is to provision `ktha-node` directly on VMs, managed by configuration tools like Salt or Ansible, rather than through container orchestration.

9. Beyond the Prototype

Per-app configuration. The current prototype uses global resource limits and idle timeouts. The architecture already supports per-app values — the controller passes configuration to the container, which translates it to runner flags. This is 10–20 lines of code away. Similarly, cgroup CPU and I/O weights (`cpu.weight`, `io.weight`) could be set per-app to allow premium tiers. Currently all guest cgroups are siblings under the same parent and default to equal weight, which provides fair sharing without explicit configuration.

Lifecycle policies. Auto-restart on crash would keep an app warm after a failure, avoiding a cold start on the next request. Crash-loop detection (e.g., 3 crashes in 60 seconds marks the app unhealthy) would prevent a broken app from consuming resources in a restart loop. The upgrade path could be made stricter: a true stop-first policy guarantees no overlap but introduces downtime; a start-first policy with a readiness gate on the new container provides honest zero-downtime upgrades. The policy could be selected per-app or even

per-upgrade as an enum argument, triggering slightly different controller methods. These are policy changes in the controller's state machine, not architectural changes.

Stronger isolation. User namespaces would allow the container to run as root inside the namespace while mapping to an unprivileged UID on the host, reducing the blast radius of a container escape. `pivot_root` would replace `chroot` for proper mount tree isolation: unlike `chroot`, which only changes the root path lookup, `pivot_root` swaps the mount tree entirely and allows unmounting the old root, making it genuinely inaccessible.

Image management. A proper image API would accept tarball uploads, unpack and validate them, and store them for deployment. Currently images are pre-built directories placed on the host filesystem.

Persistent storage. Apps currently have an ephemeral filesystem that is destroyed with the container. Persistent storage could be implemented by bind-mounting a per-app data directory into the container rootfs, separating app state from the image.

OverlayFS. The current implementation copies the entire image directory to a new rootfs for each container (`os.CopyFS`). OverlayFS would mount the image as a read-only lower layer with a per-container writable upper layer. This has three benefits. First, it replaces a full file copy with a mount operation, reducing cold start time. Second, it enables page cache sharing: when Node.js loads `index.js`, the kernel caches the file contents in memory — with copied files, each container gets separate cache entries for identical content, but with a shared lowerdir, all containers on the same image share one cache entry. Memory, not disk, is the expensive resource in cloud environments. Third, OverlayFS is the most natural foundation for persistent storage — the writable upper layer can be preserved across container restarts.

Multi-host scaling. The architecture is single-host by design. Scaling out would add a gateway layer that maps app IDs to node IDs and tracks node resource utilization. Migration would be straightforward: start the app on the new node, reroute traffic at the gateway, stop on the old node. This lays cleanly on top of the existing controller/container decomposition.

Container egress. Containers currently have no network access. Enabling outbound connections (for database access or external APIs) would require creating a veth pair between the host and container network namespaces, with iptables NAT for masquerading. For local services like databases, an alternative is passing a Unix socket into the container — avoiding the networking stack entirely.

Node.js optimization. Custom Node.js builds and runtime flags were explored during the RSS investigation. The gains were marginal — the shared page approach proved more effective.